

```
import cv2 # Import OpenCV for image processing algorithms
import numpy as np # Import Numpy for mathematical matrix o
import matplotlib.pyplot as plt # Import Matplotlib to visualize the out

# Step 1: Create a high frequency test pattern (sharp diagonal lines)
img = np.zeros((100, 100), dtype=np.uint8) # Create a blank black image array filled with zeros
for i in range(-100, 100, 10):
    cv2.line(img, (i, 0), (i+100, 100), 255, 2) # Draw a straight line on the image

# Step 2: Compress the image aggressively in-memory (JPEG Quality = 5/100)
# This forces massive quantization and chroma subsampling mathematically
encode_param = [int(cv2.IMWRITE_JPEG_QUALITY), 5]
result, encimg = cv2.imencode('.jpg', img, encode_param) # Encode image buffer into compressed forma

# Step 3: Decode back to pixels to view the compression artifacts
decimg = cv2.imdecode(encimg, 0) # Decode a compressed buffer back into image array

# Step 4: Show the visual loss caused by optimizing storage
plt.figure(figsize=(8,4)) # Create a new figure window for display
plt.subplot(121), plt.imshow(img, cmap='gray'), plt.title('Original (High Data)') # Place this image
plt.subplot(122), plt.imshow(decimg, cmap='gray'), plt.title('JPEG Artifacts (Low Data)') # Place th
plt.show() # Show the final plot window to the user
```



Start coding or [generate](#) with AI.

